CSI BISHOP SOLOMON DORAISAWMY COLLEGE OF ARTS AND SCIENCE

Department of Computer Science

Academic Year: 2023 - 2026

Project Documentation

Complete Technical Report

Enforcing Mandatory Fields Using UI Policies andMigrating Changes with Update Sets

Team Leader	KARTHIKEYAN M
Team Member 2	BALAMURUGAN K S
Team Member 3	DHANUSH P
Team Member 4	MOHAMED ISMAIL S
Platform	ServiceNow

ABSTRACT

This project demonstrates the technical implementation of dynamic data validation within the **ServiceNow** platform using **UI Policies**, coupled with a robust deployment strategy using **Update Sets**. In enterprise service management, maintaining data integrity is critical for accurate reporting and efficient workflow orchestration. The core objective of this initiative was to automate the enforcement of the **Priority** field as a mandatory requirement specifically when an **Incident** record reaches the **In Progress** state.

The technical execution was divided into two primary phases:

1. **Logic Configuration**: Developing a non-scripted UI Policy on the incident table to trigger field restrictions based on real-time state changes.
2. **Lifecycle Management**: Utilizing the **Update Set** framework to capture these configuration changes into a portable XML package, facilitating a seamless "Develop-to-Production" migration.

By successfully previewing, committing, and validating these changes across separate ServiceNow instances, this project highlights a best-practice approach to system administration that ensures data completeness without the overhead of custom scripting.

INTRODUCTION

In modern IT Service Management (ITSM), data integrity is the cornerstone of effective decision-making and operational efficiency. A common challenge faced by ServiceNow administrators is ensuring that critical fields—such as **Priority**—are populated accurately during the lifecycle of an incident. Without automated enforcement, inconsistent data entry can lead to poor service level agreement (SLA) tracking and mismanaged technician workloads.

This project provides a professional solution by leveraging **UI Policies** to dynamically control form behavior based on record status. By configuring a policy that triggers specifically when an incident is "In Progress," we mandate the selection of a Priority level without the need for complex client-side scripting.

Furthermore, to align with enterprise-grade deployment standards, this project utilizes **Update Sets**. This framework allows administrators to package configuration changes into a portable XML format, ensuring that logic tested in a Development environment can be migrated to Production with 100% fidelity and zero manual rework.

Core Project Objectives

- **Automated Validation**: Implement a UI Policy on the `incident` table to enforce data entry.
- **Conditional Logic**: Define specific "When to Apply" criteria (State = In Progress) to ensure a seamless user experience.
- **Configuration Portability**: Capture all changes within a named Update Set for secure environment migration.
- **Deployment Verification**: Demonstrate the end-to-end process of exporting, importing, previewing, and committing changes in a target instance

1. Milestone 1: Update Sets

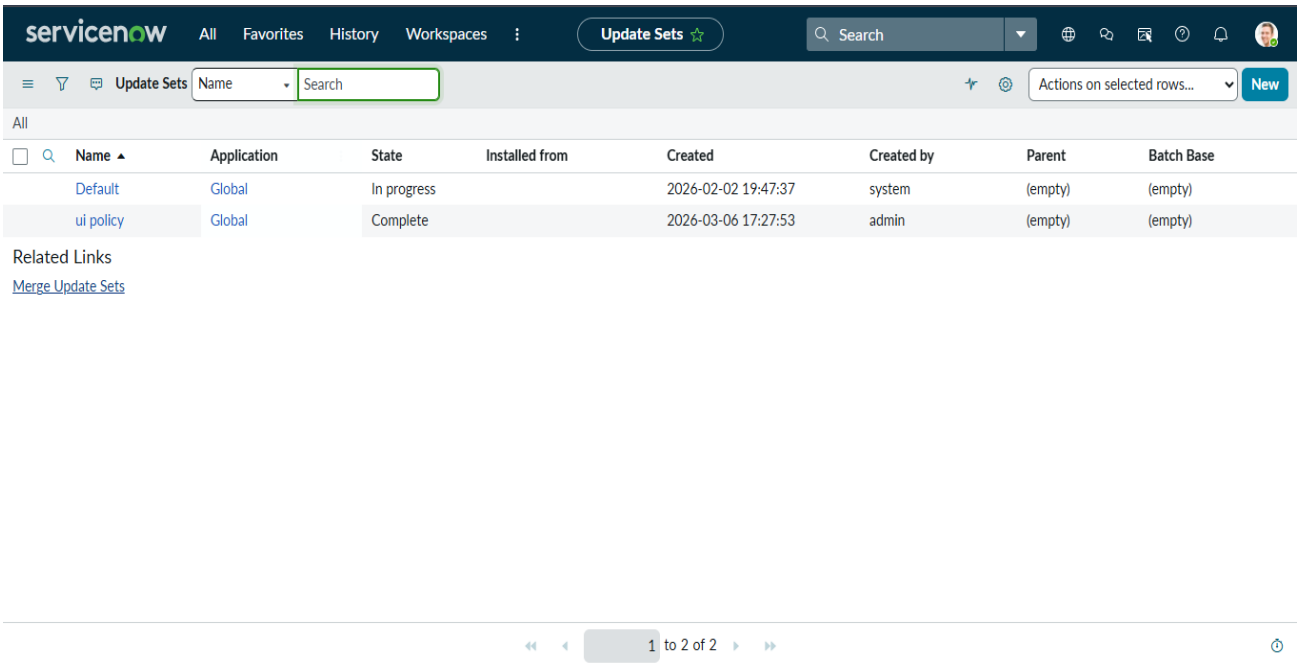
1.1 Activity 1: Create Update Set

In ServiceNow, **Update Sets** are used to capture configuration changes and move them from one instance to another. Before creating any configuration such as UI Policies, it is important to create an Update Set so that all the changes can be tracked and migrated.

In this step, a new update set is created to store the configurations related to the UI Policy that will be implemented later in the project. The update set is created by navigating to **System Update Sets → Local Update Sets** and selecting the **New** option. After entering the required details such as the name and description, the update set is saved and marked as **Current**.

By setting the update set as current, all the configuration changes made in the system will be automatically recorded in this update set. This helps in exporting and migrating the configurations to another ServiceNow instance.

created update stes:



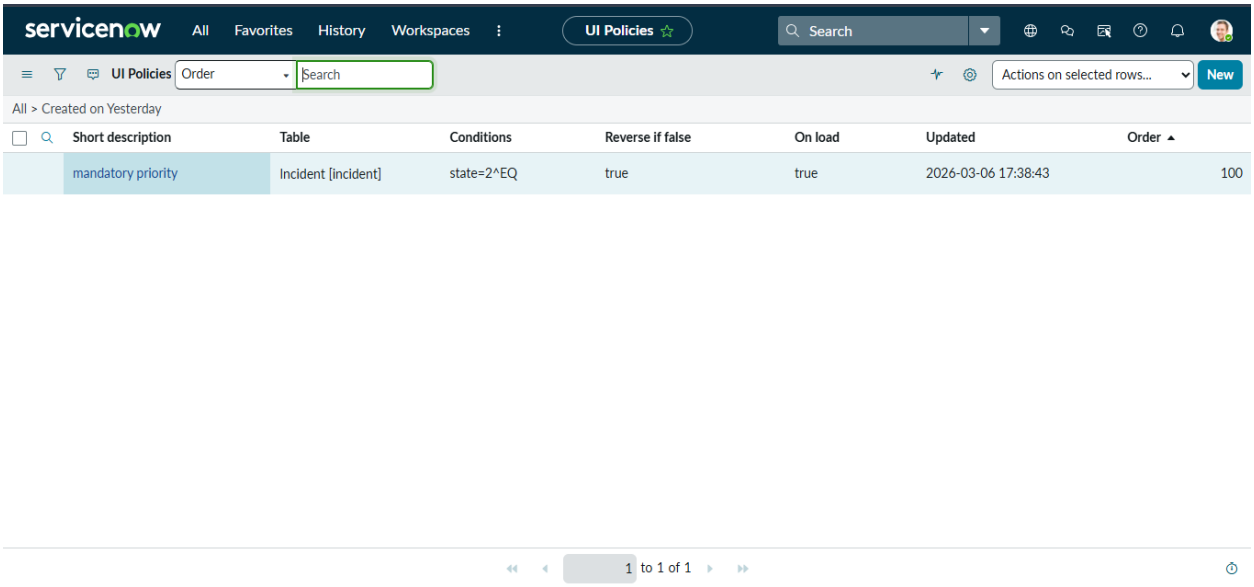
2. MILESTONE 2: UI POLICY

2.1 ACTIVITY 1: CREATE UI POLICY

A **UI Policy** in ServiceNow is used to dynamically control the behavior of form fields without writing scripts. It helps administrators enforce business rules such as making fields mandatory, read-only, or visible based on certain conditions.

In this project, a UI Policy is created to make the **Priority field mandatory when the State of the Incident is set to "In Progress"**. This ensures that the priority of the incident must be specified before the incident can proceed further in the workflow.

The UI Policy is created by navigating to **System UI → UI Policies** and selecting the **New** option. The required details such as the table name, short description, and condition are entered. After saving the UI Policy, a **UI Policy Action** is added to make the **Priority field mandatory**.

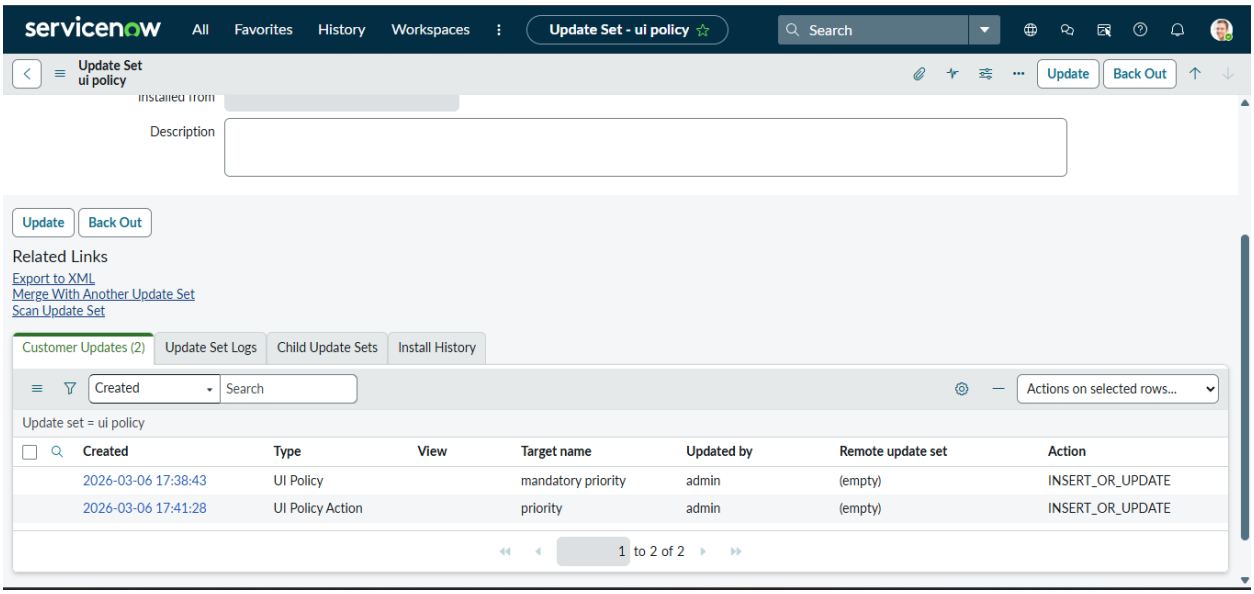


2.2 ACTIVITY 2: CHECK UPDATE SET

After creating the UI Policy, it is necessary to verify whether the changes are properly captured in the current update set.

To do this, navigate to **System Update Sets → Local Update Sets** and open the update set that was created earlier. Inside the update set, the **Customer Updates** section displays all the configuration changes recorded in that update set.

Here, the created **UI Policy** and **UI Policy Action** can be seen listed in the update set. This confirms that the changes have been successfully captured and are ready to be exported.



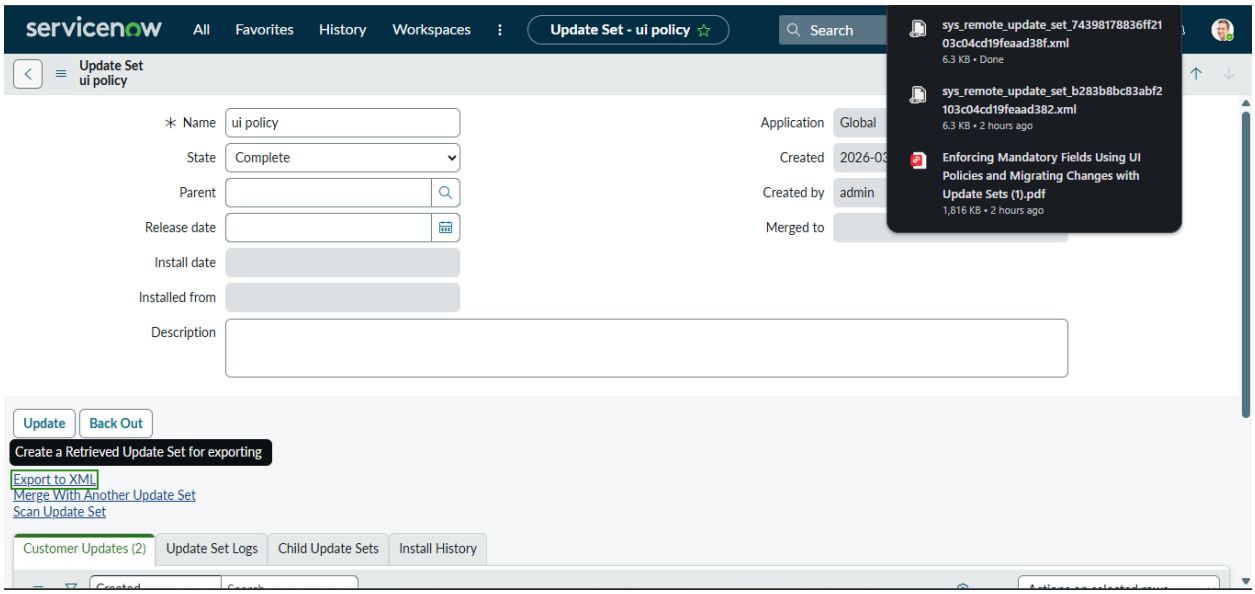
2.3 ACTIVITY 3: EXPORT AND IMPORT THE UPDATE SET

To move the configuration from one ServiceNow instance to another, the update set must be exported and then imported.

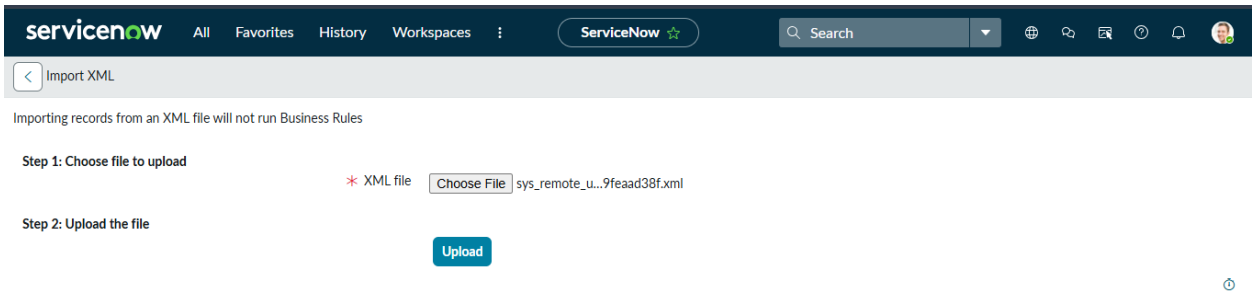
First, the state of the update set is changed to **Complete**. After that, the **Export to XML** option is used to download the update set as an XML file. This file contains all the configuration changes recorded in the update set.

Next, the XML file is imported into another ServiceNow instance by navigating to **System Update Sets → Retrieved Update Sets** and selecting **Import Update Set from XML**. After uploading the file, the update set is previewed to check for any conflicts and then committed to apply the changes in the new instance.

EXPORT:



IMPORT:



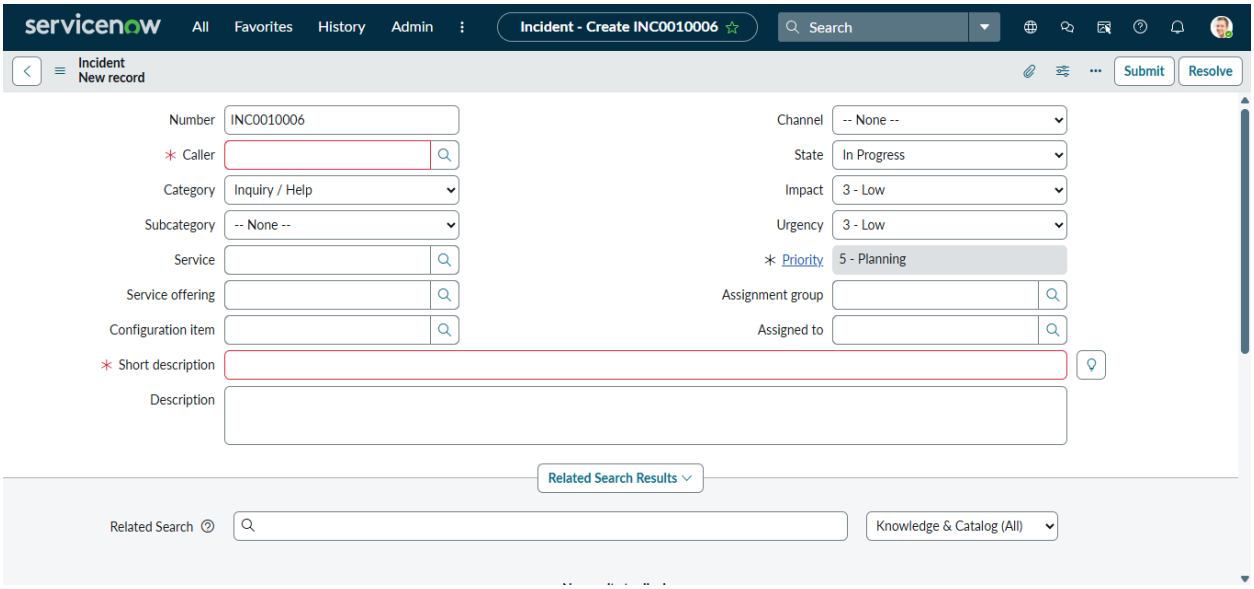
MILESTONE 3: RESULT

2.4 ACTIVITY 1: TESTING INCIDENT

After successfully committing the update set, the implemented UI Policy is tested in the **Incident module**.

An existing incident is opened or a new incident is created. When the **State** field is changed to **In Progress**, the system automatically makes the **Priority field mandatory**.

This ensures that the user cannot proceed without selecting a priority value for the incident, thereby maintaining data completeness and improving incident management workflow.



CONCLUSION

This project demonstrates how **UI Policies in ServiceNow** can be used to enforce business rules without writing any scripts. By creating a UI Policy, the **Priority field is automatically made mandatory when the Incident State is set to "In Progress."** This helps ensure that important information is captured before the incident continues in the workflow.

The project also shows how **Update Sets** can be used to track configuration changes and migrate them between different ServiceNow instances. By exporting the update set as an XML file and importing it into another instance, the configurations can be easily transferred without manually recreating them.

Overall, this project improves **data accuracy, workflow efficiency, and configuration management** in ServiceNow. It also provides practical knowledge of using **UI Policies and Update Sets**, which are important features for ServiceNow administrators and developers.